

Parallel Programming with OpenMP – part2

Academic year 2025-2026

Prof. Massimo Torquati
massimo.torquati@unipi.it

Outline

- Understand the tasking model
- Learn how to create, synchronize, and orchestrate tasks
- Use `depend` and `taskloop` to express pipeline and irregular parallelism

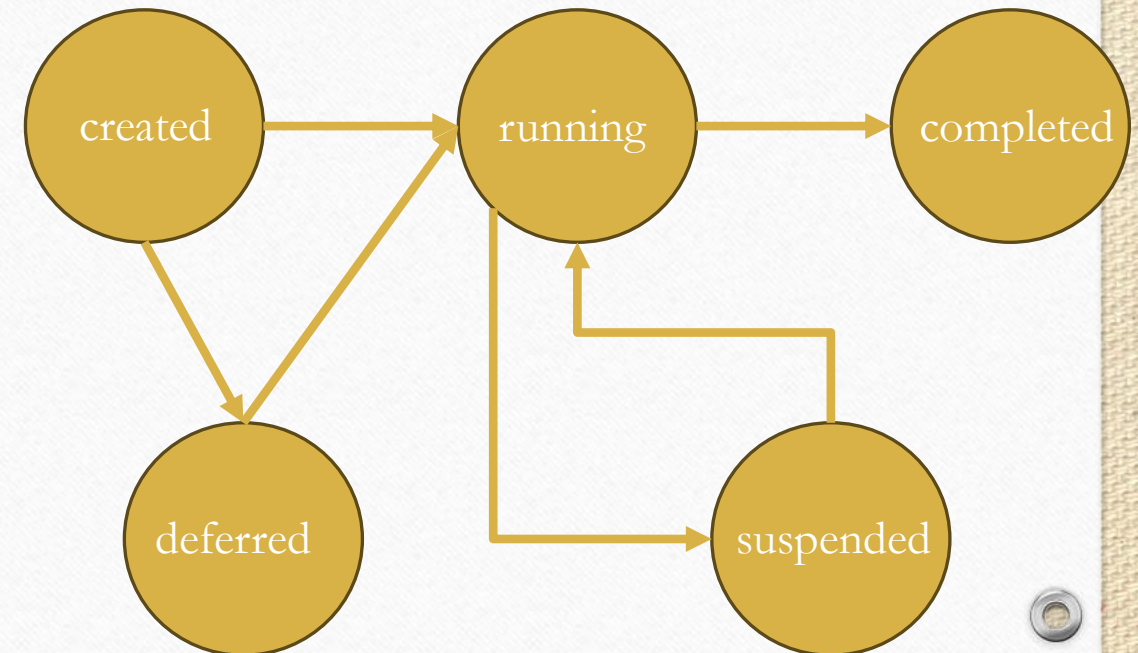
OpenMP tasks, motivations

- Even if loops are the main sources of parallelism, not all programs have easily parallelizable loops
 - Loops in which the iteration count is not known in advance
 - Loops containing complex dependencies (e.g., LU, Cholesky algorithms)
- Divide and Conquer algorithms (e.g., quicksort, merge-sort algorithms) and data streaming computations (i.e., producer-consumer pattern) can be challenging to parallelize with parallel loop or sections
- Creating tasks with the **task** directive is more flexible than **sections**
 - Sections are defined statically, while tasks can be dynamically created
 - Since OpenMP 4, it is possible to specify **depend(in/out/inout)** clauses on tasks to orchestrate task dependencies
- Tasking was a major functionality enhancement introduced in OpenMP 3.0

OpenMP tasks

- In OpenMP, a task is an **independent unit of work** created inside a parallel region **that can run immediately or later**, possibly by a different thread
 - Deferred tasks live in a task pool managed by the OpenMP runtime
- Task scheduling happens at **scheduling points**
 - However, runtime scheduling decisions can be influenced by **dependencies** and **priorities**
- An OpenMP task is composed of:
 - The code to execute
 - Its data environment, including shared and private variables
 - Task-related control information managed by the runtime

Task life-cycle



Task Scheduling points

- Task scheduling points are points in the program where the OpenMP runtime can schedule tasks for execution
 - At a task scheduling point, a thread of the team may suspend the execution of the current task and execute another task
 - Threads in the team pick a task from the task pool when they reach a scheduling point
- In a nutshell, a scheduling point can be:
 - When creating an explicit task
 - After completing the structured block of a task
 - During synchronization (e.g., implicit or explicit barriers)
 - (When offloading to a target device)

OpenMP tasks

- **Implicit tasks**

- Created automatically at the start of a **parallel** region, one per thread
- The initial task exists before any parallel region on the primary thread
- Execute immediately on their owning thread, and act as the parent of tasks created inside
- At barriers (explicit or implicit), each implicit task waits for its child tasks

- **Explicit tasks**

- Created explicitly by the **#pragma omp task** inside a running task. If many threads reach the code containing the directive, many tasks are created
- May run on any thread of the team, not ordered by source position
 - **depend(in/out/inout)** clauses may be used to order tasks through data dependences
- There are directives to wait for direct children (**taskwait**) or wait for all descendants (**taskgroup**)

OpenMP task directive

```
#pragma omp task [clause list]
```

- Some clauses in [clause list] :
 - **if (scalar expression)**: if false, the region is executed immediately as an included task
 - **final (scalar expression)**: if true, the task is marked final. Tasks created inside a final task are included tasks
 - **untied**: no scheduling restrictions for the task. The task may suspend and resume on a different thread A task, by default, is tied to the first thread that executes it.
 - **private (variable list)**: each task gets its own uninitialized private copy
 - **firstprivate (variable list)**: each task gets its own private copy initialized from the value in the generating context
 - **shared (variable list)**: specifies that variables are shared with the parent task
 - **default (data scoping specifier)**: default data handling specifier may be **shared** or **none**
 - **depend(kind: variable list)**: specifies dependencies between tasks using memory locations. The **kind** can be **in**, **out**, or **inout**, defining that the task reads, writes or reads and writes the memory locations

«Hello task» example

- In this example, the parallel region starts 2 threads, each thread runs its implicit task
- Inside that implicit task, the loop has 2 iterations, each of which creates one explicit task
- In total we have 4 tasks executed by the team
- At the end of the parallel region there is a barrier, so all 4 tasks have to finish before exit
- Maybe this is not what we want
- How can we create exactly 2 tasks, which will be executed by the 2 threads of the team?

```
hello-task1.cpp > ...
1  #include <stdio>
2  #include <omp.h>
3
4  int main() {
5
6  #pragma omp parallel num_threads(2)
7      {
8          // Each thread of the team runs its implicit task and, inside
9          // the loop, creates two explicit child tasks
10         // In total 4 tasks are created
11         for (int i=0; i<2; ++i) {
12             #pragma omp task
13             std::printf("Hello Task %d from thread %d\n",
14                         i, omp_get_thread_num());
15         }
16     }
17 }
18
```


«Hello task» example revised

- To guarantee each task is generated only once, task creation may be embedded in a **single** construct
- One thread of the team creates the tasks while other threads start executing them
- Usually, **single** is used with **nowait** to avoid an extra barrier when it is not needed

```
hello-task2.cpp > ...
1  #include <stdio>
2  #include <omp.h>
3
4  int main() {
5
6  #pragma omp parallel num_threads(2)
7      {
8          // Only one thread enters this block and creates the two tasks
9          // The other thread does not wait here and can immediately start
10         // executing the created tasks
11         #pragma omp single nowait
12         {
13             for (int i=0;i<2;++i) {
14                 #pragma omp task
15                 std::printf("Hello Task %d from thread %d\n",
16                             i, omp_get_thread_num());
17             }
18         } // <- with nowait, no implicit barrier here
19     } // <- implicit barrier, all tasks completed here
20 }
```

OpenMP tasks, execution model

- When a team thread encounters a **task** directive, it may choose to **execute the task immediately** or **defer its execution**
- If the task execution is deferred, **the task is placed in a task queue** associated with the current **parallel region**
- All threads of the team will take tasks out of the task pool until the pool is empty
- The code associated with the task construct will be executed only once
- **By default, tasks are tied to the thread that starts them. They can suspend only at scheduling points and must resume on the same thread**
- We can mark a task with the **untied** clause `#pragma omp task untied`
- An **untied** task **can be rescheduled**; it can start on one thread, get suspended, and resume on another thread of the team

Example

```
#pragma omp parallel
{
  #pragma omp task
  taskF();
  #pragma omp barrier
  #pragma omp single nowait
  {
    #pragma omp task
    taskH();
    #pragma omp task
    taskG();
  } // <- no barrier here
} // <- implicit barrier
```

- The taskF() task created for each thread in the parallel region
 - All tasks are tied to a thread in this case
- All tasks synchronize at the barrier
- Only one thread creates an instance of the taskH() task and one instance of the taskG() task. These tasks will be executed by some thread in the team in some order
- All tasks are completed before the implicit barrier at the end of the region can be crossed

Data scoping for tasks

- Data scoping for tasks is tricky if the scope is not explicitly specified
- Variables from the enclosing context are often **firstprivate** in tasks, unless they are already **shared** in the enclosing region
- This is important **because task execution may be deferred, and the original variables may no longer be valid when the task runs**
- In practice, prefer explicit **shared(...)** and **firstprivate(...)** clauses

```
int main() {  
    int x = 1;  
    int y = 2;  
    #pragma omp parallel private(y)  
    {  
        int z = 2;  
        #pragma omp task  
        {  
            int w = 3;  
            // x shared, y and z become firstprivate, and w private  
            // NOTE: y is not initialized here!  
            F(x,y,z,w);  
        }  
    }  
}
```

```
// safe variant  
#pragma omp parallel firstprivate(y) shared(x)  
{  
    ...  
    #pragma omp task firstprivate(y) shared(x,y)
```


Wait for the completion of tasks

- The thread **barrier** waits for all threads to arrive, and each thread also waits for the completion of its own child tasks. This is also true for implicit barriers at the end of a **parallel** region
- However, since child tasks are executed separately from the generating tasks, **it is possible that a child task gets executed after the generating task has finished**
- The **taskwait** directive can be used to wait for the completion of child tasks defined in the current task (direct child tasks)
 - It waits for the generated tasks, **but not for all descendant tasks**
 - The **taskwait** suspends the parent task until all its child tasks have finished
- The **taskgroup** directive defines a region in which any tasks created become part of that group. **At the end of the taskgroup region, there is an implicit barrier**
- The **taskgroup** waits for all tasks generated within its structured block (including nested tasks)

```
#pragma omp taskwait
```

```
#pragma omp taskgroup
```

taskwait vs taskgroup

- **taskwait**: the current task waits for all its direct tasks created so far. It does not wait for grandchildren
- **taskgroup**: waits for all tasks created inside the block, including their descendants. Tasks created outside the block are not waited

```
#pragma omp parallel
#pragma omp single nowait
{
    #pragma omp task
    background_work();
    for (auto x: V) {
        #pragma omp task firstprivate(x)
        compute1(x);
        #pragma omp taskwait // waits all direct children created so far
                             // including the background_work task

        compute2(x)
    }
}
```

```
#pragma omp parallel
#pragma omp single nowait
{
    #pragma omp task
    background_work();
    for (auto x: V) {
        #pragma omp taskgroup
        {
            #pragma omp task firstprivate(x)
            compute1(x);
        } // waits all tasks created in the block, including descendant
        compute2(x);
    }
} // only here the background_work task must be completed
```


Final vs if clauses

- It is possible to control the generation of tasks
- The **final** and **if** clauses are similar but with different effects
 - **if(cond)**: if cond evaluates to false, the region is executed immediately by the current thread (included task); **no deferred task is created**
 - **final(cond)**: the task **is created** and marked as final if the condition holds; descending tasks are executed inside the task (included tasks)
- The number of tasks created is different and this may have an impact on performance

```
#pragma omp task if(false)
{
    // the current thread executes taskA()
    taskA();
    #pragma omp task
    taskB();           // taskB() is a regular task and
                        // can be deferred
}

#pragma omp task final(true)
{
    // the new task is marked as final
    taskA();
    #pragma omp task
    taskB();           // taskB() become an included task
                        // it is executed within the final task context
}
```

Fibonacci example

- It computes the n-th Fibonacci number using the exponential algorithm.
- The task is the fib(n) function
- At line 8 and 10, n1 and n2 **must** be **shared**, otherwise the default would be **firstprivate**
- The **taskwait** at line 13, suspends the parent task until both the two children tasks complete. If we had not used **taskwait**, n1 and n2 would have got lost
- To stop recurring, and decreasing the overhead of task creation, we can use a cutoff value to switch to serial computation.

```
omp_fibo.cpp > ...
1  #include <stdio>
2  #include <string>
3
4  long fib( long n ) {
5      long n1, n2;
6      if ( n < 2 ) return 1;
7
8      #pragma omp task shared(n1)
9          n1 = fib(n-1);
10     #pragma omp task shared(n2)
11         n2 = fib(n-2);
12     // Wait for the two tasks to complete
13     #pragma omp taskwait
14     return n1 + n2;
15 }
16
17 int main(int argc, char *argv[]) {
18     if (argc!=3){
19         std::printf("Use %s n #threads\n", argv[0]);
20         return -1;
21     }
22     long n    = std::stol(argv[1]);
23     long nth  = std::stol(argv[2]);
24
25     long result;
26     // creating a pool of nth threads
27     #pragma omp parallel num_threads(nth)
28     {
29         // only the Master thread starts the recursion
30         #pragma omp master
31             result = fib(n);
32     }
33     std::printf("fib(%ld) = %ld\n", n, result);
34 }
```


Task scheduling

- OpenMP defines the following task scheduling points:
 - When a task is created (`#pragma omp task`)
 - When encountering `taskwait`, or `taskyield`
 - At explicit or implicit barriers
 - When a task completes execution
 - When exiting a `taskgroup` region
- **Tied tasks** (only the thread to which the task is tied may execute the task)
 - A task can be suspended at specific scheduling points (creation, completion, `taskwait`, `barrier`, ...)
 - If a task is not suspended at a barrier, the thread executing the task can only switch to a descendant of any task tied to the thread
- **Untied tasks** (the task can be executed by different threads)
 - Untied tasks have fewer scheduling restrictions: after suspension at task scheduling points, they may resume on a different thread

Task scheduling refers to the mechanism by which the OpenMP runtime system decides **which tasks** run and **when** they run on the available threads

Clauses such as the following influence the scheduling behavior:

- **if**: to decide whether a construct should be executed as a task or inline
- **final**: to mark a task as non-deferrable
- **priority**: to influence the scheduling order
- **untied**: to allow tasks to be resumed on a different thread

Task generation

- To generate task from a loop we could do something like 
- If NTASKS is very large, the implementation can stop generating new tasks and switch all threads in the team to execute the already generated tasks. If the thread that generated the tasks is executing a long task, the other threads might not have anything to do
- One option is to make the task generation an **untied** task, thus any other thread is eligible to resume the task generating the loop:

```
....  
#pragma omp single  
{  
    #pragma omp task untied  
    for(int i=0; i < NTASKS; ++i)  
        #pragma omp task  
        { ...  
}
```

```
#pragma omp parallel  
{  
    ....  
    #pragma omp single nowait  
    {  
        for(int i=0; i < NTASKS; ++i)  
            #pragma omp task  
            {  
                processTask(...);  
            }  
        } // single  
    ....  
} // parallel
```

- However, this is not flexible enough, a better option is to use **taskloop**

Taskloop

- The **taskloop** construct is designed for parallelizing loops using tasks
 - Automatically generates tasks from loop iterations, enabling dynamic scheduling
 - Useful when dynamic scheduling is needed or **iteration count is not predictable**
- Decompose a loop into chunks and create one task for each chunk
 - The overhead with respect to **#pragma omp parallel for** is generally higher due to task creation
 - Useful to integrate loops into a larger task DAG
- Some clauses in **[clause list]**:
 - **grainsize**(**grain-size**): chunks have at least **grain-size** iterations
 - **num_tasks**(**num-tasks**): create **num-tasks** chunks, each must have at least one iteration
 - They are mutually exclusive
- If neither **grainsize** nor **num_tasks** is used, the number of chunks and iterations per chunk is implementation dependent
- The **taskloop** creates one explicit task per chunk of iterations. By default, it behaves as if enclosed in a **taskgroup**, so there's an implicit wait at the end of the loop until all those tasks (and their descendants) complete
 - It is possible to use the **nogroup** clause to remove the implicit wait

```
#pragma omp taskloop [clause list]
```

```
#pragma omp parallel
{
    #pragma omp single nowait
    {
        #pragma omp taskloop
        for(int i=0; i < N; ++i)
            do_something(i);
    }
}
```

Task reduction

- In a **taskgroup** it is possible to declare the reduction object with the clause **task_reduction(op: var)**
- Every contributing **task** (or **taskloop**) should use **in_reduction(op: var)** so its partial value is combined into the object
 - The reduced variable must be **shared** in the contributing tasks
- The final value is available when the **taskgroup** ends
- Initialize the variable to the identity before the group
- OpenMP 5.0 lets you write **reduction(+: var)** directly on **task** and **taskloop**, and adds more options

```
omp_task_reduction.cpp > ...
1  #include <stdio>
2  #include <omp.h>
3
4  int main(void) {
5      double sum = 0.0;
6
7      #pragma omp parallel
8      #pragma omp single
9      {
10         #pragma omp taskgroup task_reduction(+: sum)
11         {
12             #pragma omp task in_reduction(+: sum)
13             sum += 1.0;
14
15             #pragma omp task in_reduction(+: sum)
16             sum += 2.0;
17         } // waits and combines the two contributions
18
19         std::printf("sum = %.1f\n", sum);
20     }
21 }
```


Pointer-chasing loop

- Let us consider the following loop

```
for(Node* cur=head; cur, cur=cur->next) {  
    do_something(cur);  
}
```

- We cannot use a worksharing parallel for, because the number of elements may not be known beforehand and the loop is not in canonical form

Example: omp_pchaising.cpp

```
void do_something(Node*);  
  
const int tasksize = 64; // choosed task granularity  
#pragma omp parallel  
#pragma omp single nowait  
{  
    Node* cur = head;  
    while(cur) {  
        Node* start = cur;  
        // advance up to tasksize-1 steps, stop early at list end  
        for(int c=1; c<tasksize && cur->next; ++c) cur=cur->next;  
        Node* next = cur->next;  
  
        #pragma omp task firstprivate(start,next)  
        for(Node* p=start; p != next; p = p->next) {  
            do_something(p);  
        }  
        cur = next;  
    }  
}
```

Finding primes with tasks

- Two versions, both using tasks with granularity tasksize
- The first one (sketched here) uses a per-task buffer
 - Pros: deterministic output order without final sort
 - Cons: many small buffers if ntasks is large
- The second one uses a per-thread buffer, a taskloop and a reduction
 - Pros: fewer buffers (just one per thread)
 - Cons: it needs a final sort if primes are printed

```
34
35 std::vector<ull> results;
36 results.reserve((std::size_t)((n2 - n1) / log(n1)));
37
38 // total number of tasks
39 const std::size_t ntasks = (std::size_t)((n2-n1 + tasksize-1)/tasksize);
40
41 std::vector<std::vector<ull>> buckets(ntasks); // per-task buffer
42
43 auto t0 = clock_type::now();
44 #pragma omp parallel
45 #pragma omp single nowait
46 {
47     #pragma omp taskgroup
48     for (std::size_t k = 0; k < ntasks; ++k) {
49         ull b = n1 + k * tasksize;
50         ull e = std::min<ull>(b + tasksize, n2);
51         #pragma omp task firstprivate(k, b, e)
52         {
53             auto& buf = buckets[k];
54             buf.reserve(256); // it is just a reasonable value
55             for (ull x = b; x < e; ++x)
56                 if (is_prime(x)) buf.push_back(x);
57         }
58     } // all tasks finished here
59 }
60 // merging in order
61 for (std::size_t k = 0; k < ntasks; ++k)
62     results.insert(results.end(), buckets[k].begin(), buckets[k].end());
63 auto t1 = clock_type::now();
64
65 std::cout << "Found " << results.size() << " primes\n";
66 std::cout << "Time (s): " << std::chrono::duration<double>(t1 - t0).count() << '\n';
67
68 if (print_primes) {
69     for (ull r : results)
70         std::cout << r << ' ';
71     std::cout << "\n\n";
72 }
```


Task dependencies

- Task dependencies allow to define fine-grained synchronization for the execution order of tasks
 - Finer control than `taskwait`, which waits on all direct children.
- The `depend` clause on a task specifies dependencies using memory locations (not mutual exclusion)

```
#pragma omp task depend(dependency-type: list)
```

- A dependent task cannot be scheduled until all earlier sibling tasks with conflicting dependencies **on overlapping storage** have completed
- The dependency-type in OpenMP 4.5
 - `in` : reads the listed storage
 - `out` : writes the listed storage
 - `inout` : reads and writes the listed storage
- What can be in the depend list: **lvalues of storage** or **array sections** (e.g., `A[0:8]`)
- More recent OpenMP versions, have added more **dependency-type** and features

Scope of dependencies:

- Dependences order sibling tasks created by the same team
- Distinct (even nested) parallel regions create different teams and do not share the task graph, so dependencies across them do not work

Example 1

- For each index i , one task produce $x[i]$ one task consume $x[i]$, and one independent task just print the index
- Produce $x[i]$ always appears before consume $x[i]$ because of the matching `depend(out : x[i])` `depend(in : x[i])`
- The independent task (lines 30-35) can print the index before, between, or after the print of the dependent tasks
- This is the classic **Read-after-Write (RAW)** dependency

```
omp_depend.cpp > ...
1  #include <stdio>
2  #include <thread>
3  #include <chrono>
4  #include <omp.h>
5
6
7  int main() {
8      const int N = 5;
9      int x[N];
10
11     #pragma omp parallel num_threads(4)
12     #pragma omp single nowait
13     for (int i = 0; i < N; ++i) {
14         #pragma omp task firstprivate(i) shared(x) depend(out: x[i])
15         {
16             std::this_thread::sleep_for(std::chrono::milliseconds(30 + 5*i));
17             x[i] = i;
18             std::printf("produced, x[%d]=%d on thread %d\n",
19                         i, x[i], omp_get_thread_num());
20         }
21
22         #pragma omp task firstprivate(i) shared(x) depend(in: x[i])
23         {
24             std::printf("consumed x[%d]=%d on thread %d\n",
25                         i, x[i], omp_get_thread_num());
26             std::fflush(stdout);
27         }
28
29         // Independent task, can run in any order
30         #pragma omp task firstprivate(i)
31         {
32             std::this_thread::sleep_for(std::chrono::milliseconds(10 + 5*i));
33             std::printf("independent task, i=%d on thread %d\n", i, omp_get_thread_num());
34             std::fflush(stdout);
35         }
36     }
37 }
```


Example 2

- In this simple example we have multiple input dependencies from a single output dependency
- The input tasks can execute in any order

```
omp_depend2.cpp > ...
1  #include <stdio>
2
3  int main() {
4      int x = 0;
5
6      #pragma omp parallel
7      #pragma omp single nowait
8      {
9      #pragma omp task shared(x) depend(out: x)
10         x = 1;
11      #pragma omp task shared(x) depend(in: x)
12         std::printf("x+1= %d\n", x+1);
13      #pragma omp task shared(x) depend(in: x)
14         std::printf("x+2= %d\n", x+2);
15      #pragma omp task shared(x) depend(in: x)
16         std::printf("x+3= %d\n", x+3);
17     }
18 }
19
```

Example 3

- Anti dependency (Write-after-Read, WAR):
 - A later task wants to write a location that an earlier task still reads
 - The writer must wait until the read is finished
 - OpenMP expresses this with `depend(in:...)` on the reader and `depend(out:...)` on the writer
- Output dependency (Write-after-Writer, WAW):
 - Two writes to the same location must be ordered

Examples: `omp_depend3.cpp`

```
omp_depend3.cpp > ...
7  int main() {
11     // WAR dependency example
12     std::printf("Write-after-Read dependency\n");
13     #pragma omp parallel
14     #pragma omp single nowait
15     {
16     #pragma omp task shared(x, y) depend(in: x)
17     {
18         std::printf("Reader (%d) starts\n", omp_get_thread_num());
19         std::this_thread::sleep_for(std::chrono::milliseconds(800));
20         y = x;
21         std::printf("Reader (%d) ends\n", omp_get_thread_num());
22     }
23     #pragma omp task shared(x) depend(out: x)
24     {
25         std::printf("Writer (%d) starts\n", omp_get_thread_num());
26         x = 2;
27     }
28     }
29     assert(y==1 && x==2);
30     x=0; y=0;
31
32     // WAW dependency example
33     std::printf("\nWrite-after-Write dependency\n");
34     #pragma omp parallel
35     #pragma omp single nowait
36     {
37     #pragma omp task shared(x) depend(out: x)
38     {
39         std::printf("Writer1 (%d) starts\n", omp_get_thread_num());
40         std::this_thread::sleep_for(std::chrono::milliseconds(800));
41         x = 1;
42         std::printf("Writer1 (%d) ends\n", omp_get_thread_num());
43     }
44     #pragma omp task shared(x) depend(out: x)
45     {
46         std::printf("Writer2 (%d) starts\n", omp_get_thread_num());
47         x = 2;
48     }
49     }
50     assert(x==2);
51 }
```


Example 4

- Array sections example
- `#pragma omp task depend(in: A[4:11])`
 - Means elements A[4], ... A[14]
- Syntax: base[lower : length]
- Sections are defined by element count, not bytes

Examples: omp_depend4.cpp

```
omp_depend4.cpp > ...
1  #include <omp.h>
2  #include <cstdio>
3
4  int main() {
5      const int N = 32;
6      int A[N];          // shared array
7
8      #pragma omp parallel num_threads(4)
9      #pragma omp single nowait
10     {
11         // Produce two disjoint halves of the array in parallel
12         #pragma omp task depend(out: A[0:16])
13         {
14             for (int i = 0; i < 16; ++i) A[i] = i;
15             std::printf("filled A[0:16] on thread %d\n", omp_get_thread_num());
16         }
17
18         #pragma omp task depend(out: A[16:32])
19         {
20             for (int i = 16; i < 32; ++i) A[i] = i;
21             std::printf("filled A[16:32] on thread %d\n", omp_get_thread_num());
22         }
23
24         // Each consumer waits only for its own section
25         #pragma omp task depend(in: A[0:16])
26         {
27             long s1 = 0; for (int i = 0; i < 16; ++i) s1 += A[i];
28             std::printf("sum A[0:16] = %ld on thread %d\n", s1, omp_get_thread_num());
29         }
30
31         #pragma omp task depend(in: A[16:32])
32         {
33             long s2 = 0; for (int i = 16; i < 32; ++i) s2 += A[i];
34             std::printf("sum A[16:32] = %ld on thread %d\n", s2, omp_get_thread_num());
35         }
36
37         // This task needs the whole array, so it waits for both producers
38         #pragma omp task depend(in: A[0:32])
39         {
40             long tot = 0; for (int i = 0; i < 32; ++i) tot += A[i];
41             std::printf("sum A[0:32] = %ld on thread %d\n", tot, omp_get_thread_num());
42         }
43     }
44 }
```

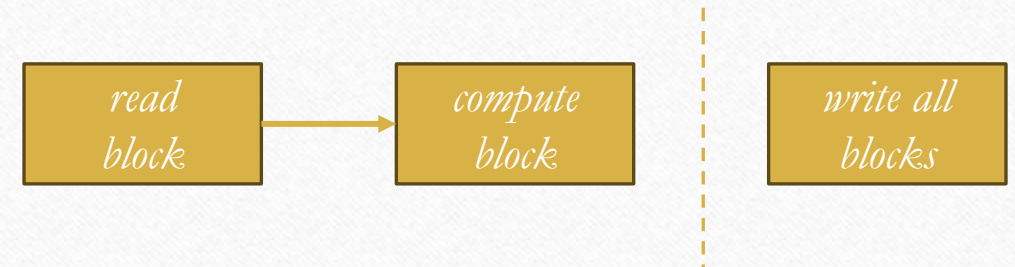
Waiting with taskyield

- taskyield creates a **scheduling point**. The runtime may suspend the current task and schedule a different ready task from the same team, then later resume the yielding task
- The runtime is allowed to ignore a yield hint (most implementations do not ignore it)
- Useful in polling loops inside tasks, and in long-running tasks to let other ready tasks make progress
- With **tied** tasks, the yielding task will later resume on the same thread. With **untied** tasks, it may resume on a different thread

```
14 int main(int argc, char** argv) {
15     const int use_yield = (argc > 1);
16     const int ntasks = 10;
17
18     omp_lock_t L;
19     omp_init_lock(&L);
20
21     #pragma omp parallel
22     #pragma omp single nowait
23     {
24         for (int i = 0; i < ntasks; ++i) {
25             #pragma omp task firstprivate(i)
26             {
27                 busy_ms(5); // do something outside the critical section
28                 int spins = 0;
29
30                 // Try-lock loop with optional taskyield
31                 while (!omp_test_lock(&L)) {
32                     ++spins;
33                     if (use_yield) {
34                         #pragma omp taskyield
35                     }
36                 }
37                 // Critical part held for a short time to make contention visible
38                 std::printf("task %d in critical on thread %d, spins=%d\n",
39                             i, omp_get_thread_num(), spins, use_yield ? " (yield)":"" );
40                 busy_ms(8); // do something in the critical section
41                 omp_unset_lock(&L);
42             }
43         } // for
44     } // single and parallel
45
46     omp_destroy_lock(&L);
47 }
```


Pipeline with Overlapped I/O

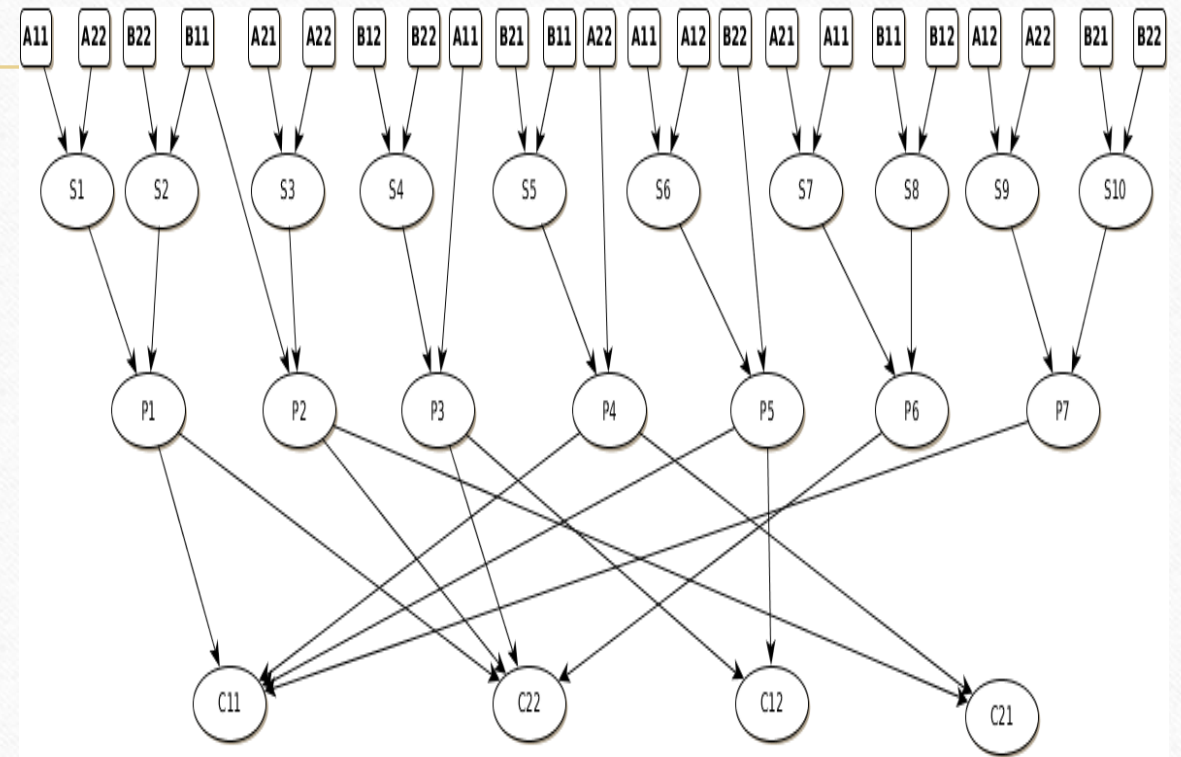
- Logical 3-stage pipeline: read->compute->write
- First version: pure map on file blocks
 - Pipeline with tasks, all phases overlaps
 - write can run in parallel with read and compute
 - Possible extension: merge read, compute, and write in a single task (left as an exercise)
- Second version: the size of the output block is not the same as the input block, writes must be executed in order
 - Pipeline with tasks for read and compute
 - Exclusive scan to compute the offset of each block
 - Parallel for to write blocks in parallel
- **NOTE:** parallel writes to one file may not speed up, the FS can serialize work. **Always test and measure on the target system**



The Strassen's GEMM algorithm

$$\begin{bmatrix} A11 & A12 \\ A21 & A22 \end{bmatrix} \times \begin{bmatrix} B11 & B12 \\ B21 & B22 \end{bmatrix} = \begin{bmatrix} C11 & C12 \\ C21 & C22 \end{bmatrix}$$

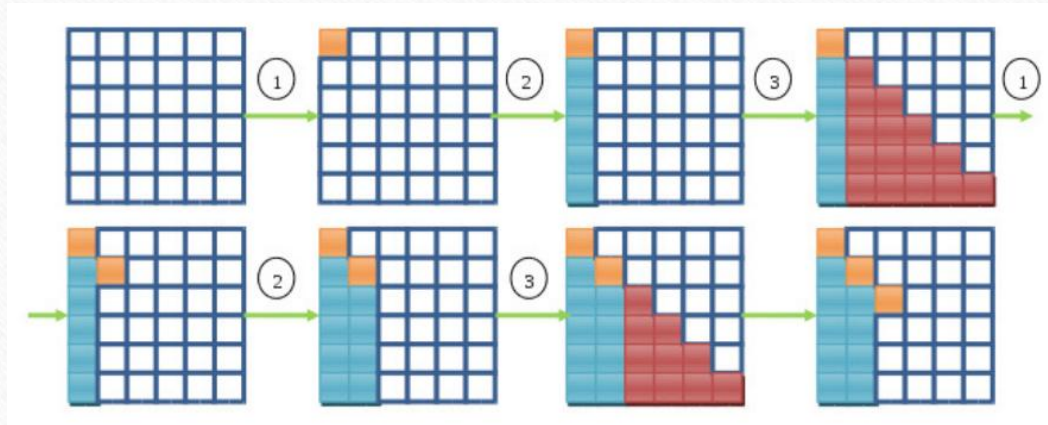
$$\begin{aligned} S1 &= A11 + A22 & S2 &= B11 + B22 & P1 &= S1 * S2 \\ S3 &= A21 + A22 & P2 &= S3 * B11 \\ S4 &= B12 - B22 & P3 &= A11 * S4 \\ S5 &= B21 - B11 & P4 &= A22 * S5 \\ S6 &= A11 + A12 & P5 &= S6 * B22 \\ S7 &= A21 - A11 & S8 &= B11 + B12 & P6 &= S7 * S8 \\ S9 &= A12 - A22 & S10 &= B21 + B22 & P7 &= S9 * S10 \\ C11 &= P1 + P4 - P5 + P7 \\ C12 &= P3 + P5 \\ C21 &= P2 + P4 \\ C22 &= P1 - P2 + P3 + P6 \end{aligned}$$



- Multiply two $N \times N$ matrices by splitting them into 4 blocks of size $\frac{N}{2}$ and doing 7 block multiplications instead of 8, and then recombining with additions and subtractions. Lower asymptotic cost, $O(N^{2.807})$ instead of $O(N^3)$
- Each macro instruction (e.g., matrix add/sub) can be computed using optimized linear algebra routines (e.g., those provided by libraries such as PLASMA, Lapack, Armadillo, ...)

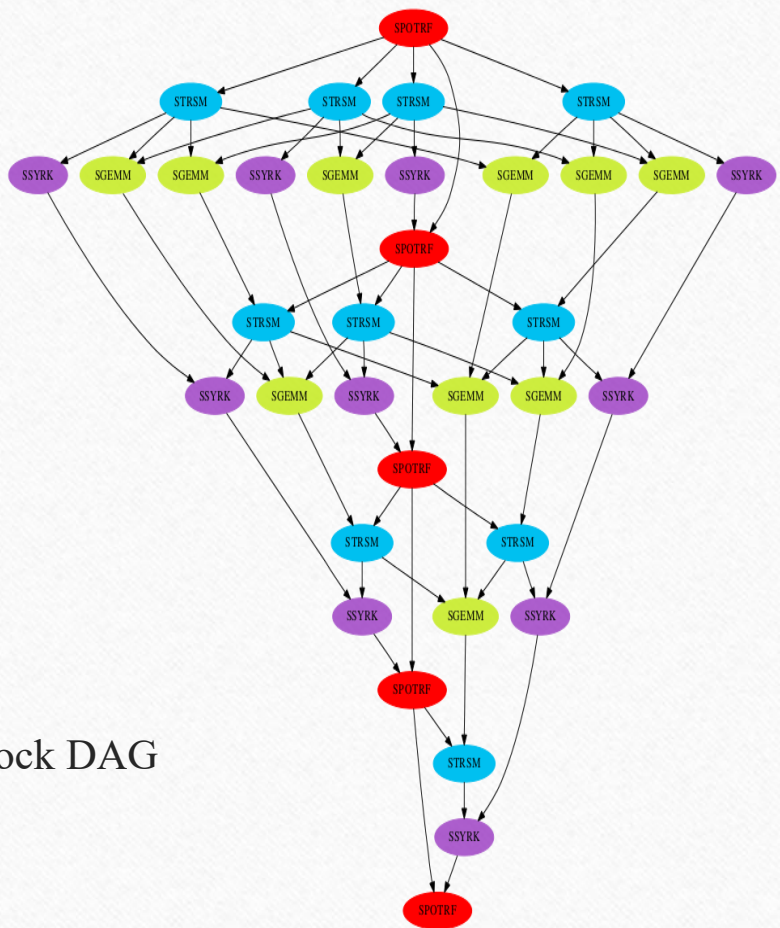
Block-Cholesky Factorization (Left-looking)

Input: $A \in \mathbb{C}^{N \times N}$ partitioned into $nb \times nb$ blocks of size $bs \times bs$, where $N = nb \cdot bs$
 Output: L such that $A = L^H L$ (stored in place in A)



```
for (int k = 0; k < nb; k++) {
    // Update the current diagonal block A[k][k]
    for (int i = 0; i < k; i++)
        F_CHERK(A_kk, A_ki, A_kk); //  $A_{k,k} \leftarrow A_{k,k} - A_{k,i} A_{k,i}^H$ 
    // Factorize the diagonal block A[k][k]
    F_CPOTF2(A_kk, A_kk); //  $A_{k,k} = chol(A_{k,k})$ 
    // Update the trailing blocks A[j][k] for k+1, ..., nb-1.
    for (int j = k+1; j < nb; j++) {
        // For each i from 0 to k-1, update A[j][k]
        for (int i = 0; i < k; i++)
            F_CGEMM(A_ki, A_ji, A_jk, A_jk); //  $A_{j,k} \leftarrow A_{j,k} - A_{j,i} A_{k,i}^H$ 
        // Finally, solve the triangular system on A[j][k]
        F_CTRSM(A_kk, A_jk, A_jk); //  $A_{j,k} \leftarrow A_{j,k} (A_{k,k}^H)^{-1}$ 
    }
}
```


Block-Cholesky in OpenMP



5-block DAG

```

103 void cholOMP(float_complex_t *A, int nb, int bs) {
104     #pragma omp parallel
105     {
106         #pragma omp single nowait
107         {
108             for (int k = 0; k < nb; k++) {
109                 // --- Step 1: Update the current diagonal block A[k][k] using CHERK.
110                 for (int i = 0; i < k; i++) {
111                     float_complex_t* A_kk = block(A, k, k, nb, bs);
112                     float_complex_t* A_ki = block(A, k, i, nb, bs);
113                     #pragma omp task firstprivate(k, i) depend(in: A_ki[0:bs*bs]) depend(inout: A_kk[0:bs*bs])
114                     F_CHERK(A_kk, A_ki, A_kk);
115                 }
116                 // --- Step 2: Factorize the diagonal block A[k][k] with CPOTF2.
117                 float_complex_t* A_kk = block(A, k, k, nb, bs);
118                 #pragma omp task firstprivate(k) depend(inout: A_kk[0:bs*bs])
119                 F_CPOTF2(A_kk, A_kk);
120                 // --- Step 3: Update the trailing blocks A[j][k] for j = k+1, ..., nb-1.
121                 for (int j = k+1; j < nb; j++) {
122                     float_complex_t* A_jk = block(A, j, k, nb, bs);
123                     #pragma omp task firstprivate(k, j) depend(in: A_kk[0:bs*bs]) depend(inout: A_jk[0:bs*bs])
124                     // For each i from 0 to k-1, update A[j][k] with CGEMM.
125                     for (int i = 0; i < k; i++) {
126                         float_complex_t* A_ki = block(A, k, i, nb, bs);
127                         float_complex_t* A_ji = block(A, j, i, nb, bs);
128                         #pragma omp task firstprivate(k, j, i) depend(in: A_ki[0:bs*bs], A_ji[0:bs*bs]) depend(inout: A_jk[0:bs*bs])
129                         F_CGEMM(A_ki, A_ji, A_jk, A_jk);
130                     }
131                     float_complex_t* A_kk = block(A, k, k, nb, bs);
132
133                     // Finally, solve the triangular system on A[j][k] with CTRSM.
134                     #pragma omp task firstprivate(k, j) depend(in: A_kk[0:bs*bs]) depend(inout: A_jk[0:bs*bs])
135                     F_CTRSM(A_kk, A_jk, A_jk);
136                 }
137             } // for k
138             #pragma omp taskwait // Ensure all tasks complete
139             } // omp single
140         } // omp parallel
141     }

```